

LIVRET TECHNIQUE : AUTO ARCHITECT +

Génération de plans par IA et construction robotisée en environnement simulé

SOMMAIRE

1. Introduction et Vision du Projet.....	3
2. Présentation et Architecture Globale du Système.....	4
3. Logiciels et technologies.....	5
4. Architecture logicielle ROS.....	6
5. Module IA 1 : Génération du Dataset.....	8
6. Module IA 1 : Prédicteur de Caractéristiques (CNN 1D).....	8
7. Moteur Géométrique : Blueprinting et YAML.....	10
8. Algorithme de Pilotage Initial : Go-to-Goal.....	11
9. Module IA 2 : Apprentissage par Renforcement (RL).....	11
ANNEXE :	13

1. Introduction et Vision du Projet

Les avancées récentes en intelligence artificielle et en robotique ouvrent de nouvelles perspectives dans de nombreux domaines, notamment celui du bâtiment. Aujourd'hui, ces technologies permettent d'imaginer des solutions innovantes pour concevoir, visualiser et construire des bâtiments de manière plus sûre et plus efficace.

C'est dans ce contexte que s'inscrit notre projet **AutoArchitect+**, proposé par notre enseignant encadrant **M. Franco**. L'objectif du projet est de développer un système capable de transformer une simple description textuelle d'une maison en un plan de construction généré par une intelligence artificielle, puis de simuler la construction de ce bâtiment à l'aide d'un robot mobile équipé d'un bras mécanique.

Réalisé entièrement en simulation, **AutoArchitect+** combine la robotique, l'IA et l'architecture afin de proposer une démonstration concrète de l'apport des nouvelles technologies dans le domaine du bâtiment. À travers des interfaces simples et intuitives, le projet vise à faciliter la compréhension, la présentation et l'analyse des résultats obtenus.

Ce projet a également pour ambition de mettre en avant les bénéfices de l'automatisation, notamment en matière de sécurité sur les chantiers, d'amélioration des conditions de travail et d'évolution des pratiques dans le secteur de la construction.

2. Présentation et Architecture Globale du Système

Le projet **AutoArchitect+** est un système intégrant une interface utilisateur, une intelligence artificielle générative et une plateforme de simulation robotique. Il vise à automatiser le processus allant de la conception d'un bâtiment à sa construction simulée.

L'utilisateur décrit le type de maison souhaité via une interface dédiée. Cette description est traitée par une IA générative qui produit un plan de construction correspondant aux besoins exprimés. Le plan généré est ensuite affiché à l'utilisateur pour validation.

Enfin, ce plan est exploité par un robot mobile équipé d'un bras mécanique, évoluant dans un environnement virtuel, afin de simuler la construction du bâtiment. L'ensemble du projet fonctionne en mode simulation, garantissant sécurité et flexibilité tout en facilitant la présentation des résultats.

Le système repose sur un flux unidirectionnel :

- ✓ **Input** : Prompt textuel.
- ✓ **Traitement** : CNN pour l'extraction de données.
- ✓ **Output Logique** : Fichiers PNG et YAML structuré.
- ✓ **Exécution** : Robot mobile sous ROS piloté par PPO (RL).

3. Logiciels et technologies

- Le système principal utilisé est : **Ubuntu 20.04 LTS** (via WSL2)

Ce choix garantit une compatibilité optimale avec ROS et les outils de simulation.

- Nous utilisons **ROS Noetic**, qui assure :

- La communication entre les différents nœuds.
- La gestion des capteurs et actionneurs.
- L'intégration avec Gazebo et RViz.

ROS constitue la colonne vertébrale du système robotique.

- Les langages utilisés sont :

- **Python** : pour le développement des nœuds ROS, des environnements d'apprentissage et des scripts d'entraînement.
- **XML / YAML** : pour la configuration ROS.

- Bibliothèques et outils utilisés

- **Stable-Baselines3** : implémentation des algorithmes RL (PPO).
- **Gym / Gymnasium** : création de l'environnement d'apprentissage.
- **NumPy** : calculs numériques.
- **Gazebo** : simulation physique.
- **RViz** : visualisation et débogage.

4. Architecture logicielle ROS

Notre projet est structuré dans un **workspace Catkin**, nommé **catkin_ws**, selon les bonnes pratiques ROS :

- **src/** : contient tous les packages et répertoires sources du projet.
- **build/** et **devel/** : générés automatiquement par **catkin_make**, ces répertoires permettent de compiler et d'initialiser l'environnement de développement.

Cette organisation facilite l'intégration, la compilation et la simulation des différents composants logiciels de notre projet.

a. Description des packages et répertoires

Dans le répertoire **src/**, le projet est organisé en trois sous-répertoires principaux :

1. **scripts/**

Contient les nœuds ROS responsables de la **construction virtuelle du bâtiment** :

- **house_tracer_node.py** : permet au robot de suivre un plan de construction sans apprentissage préalable.
- **house_tracer_node1.py** : utilise un modèle pré-entraîné par apprentissage par renforcement pour que le robot trace des lignes droites sur le plan de construction.

2. **plan/**

Contient les représentations des plans de construction dans différents formats :

- Images **.png** pour la visualisation du plan.
- Fichiers **.yaml** pour la lecture et l'interprétation programmatique par les nœuds ROS.

3. **rl/**

Contient tous les fichiers relatifs à l'apprentissage par renforcement :

- **train_ppo.py** : script d'entraînement du modèle de construction des murs droits avec PPO.
- **wall_follow_env.py** : définition de l'environnement Gym et gestion des récompenses et pénalités.
- **inference.py** : permet de charger et utiliser un modèle pré-entraîné, utilisé ensuite par **house_tracer_node1.py**.

Cette organisation modulaire permet de séparer clairement :

- Les nœuds de simulation (**scripts/**).
- Les données (**plan/**).
- L'apprentissage par renforcement (**rl/**).

b. Topics, messages et communications

Les nœuds communiquent principalement via :

- `/cmd_vel` : commandes de vitesse envoyées au robot.
- `/wall_marker` : visualisation des murs dans RViz.
- `/scan` : données Lidar pour détection des obstacles.
- `/odom` : odométrie pour suivi de position et vitesse du robot.

Les messages standards ROS (Twist, LaserScan, Odometry, Marker) sont utilisés pour assurer une communication fiable entre les nœuds.

c. Visualisation et débogage

- **RViz** : visualisation des murs et du mouvement du robot dans l'environnement simulé.
- **Gazebo** : simulation physique complète du robot et du chantier.
- **Logs ROS** : analyse des erreurs et suivi du comportement du robot pendant la simulation.

Cette architecture logicielle modulaire et claire permet de gérer efficacement la simulation de la construction automatisée d'un bâtiment, avec ou sans apprentissage par renforcement.

5. Module IA 1 : Génération du Dataset

Pour entraîner le modèle, un dataset de **10 000 entrées** a été généré.

- **Logique** : Utilisation de templates aléatoires pour varier la syntaxe.
- **Variables** : Nombre de chambres (1-6), SDB (salles de bain) (1-4), Salon (20-60m²), Cuisine (10-30m²).
- **Utilité** : Permet au modèle de devenir robuste aux synonymes ("habitation", "maison", "logement").

```
def generate_house_prompt(nb_ch, nb_sdb, salon, cuisine):
    variations = [
        f"maison avec {nb_ch} chambres, {nb_sdb} salles de bain, un salon de {salon} m² et une cuisine de {cuisine} m²",
        f"une maison familiale comprenant {nb_ch} chambres, {nb_sdb} salles de bain, un grand salon de {salon} mètres carrés et une cuisine de {cuisine} m²",
        f"habitation avec {nb_ch} chambres, {nb_sdb} salles d'eau, salon de {salon} m² et cuisine de {cuisine} m²",
        f"une maison moderne avec {nb_ch} chambres, {nb_sdb} salles de bain, salon {salon} m² et cuisine {cuisine} m²"
    ]
    return random.choice(variations)

data = []
for _ in range(10000):
    nb_ch = random.randint(1, 6)
    nb_sdb = random.randint(1, 4)
    salon = random.randint(20, 60)
    cuisine = random.randint(10, 30)
    total = salon + cuisine + nb_ch * 15 + nb_sdb * 10
    prompt = generate_house_prompt(nb_ch, nb_sdb, salon, cuisine)
    data.append([prompt, nb_ch, nb_sdb, salon, cuisine, total])

df = pd.DataFrame(data, columns=["prompt", "nb_chambres", "nb_sdb", "surf_salon", "surf_cuisine", "surf_totale"])
df.to_csv("house_dataset_large.csv", index=False)
print(f"✅ Dataset généré : {len(df)} exemples")
df.head(5)
```

6. Module IA 1 : Prédicteur de Caractéristiques (CNN 1D)

Le choix d'un **CNN 1D** au lieu d'un RNN classique permet une extraction rapide des motifs locaux dans les phrases.

- **Embedding** : Vectorisation des mots sur 64 dimensions.
- **Convolution** : Extraction des relations entre les mots (ex: "3" associé à "chambres").
- **Régression** : Sortie linéaire pour prédire des valeurs continues.


```
# ---- Charger le dataset ----
df = pd.read_csv("house_dataset_large.csv")

# ---- Préparer les données texte ----
tokenizer = Tokenizer(num_words=5000, oov_token="<OOV>")
tokenizer.fit_on_texts(df["prompt"])
sequences = tokenizer.texts_to_sequences(df["prompt"])
padded = pad_sequences(sequences, maxlen=30, padding='post', truncating='post')

# ---- Préparer les sorties ----
y = df[["nb_chambres", "nb_sdb", "surf_salon", "surf_cuisine", "surf_totale"]].values

# ---- Division train/test ----
X_train, X_test, y_train, y_test = train_test_split(padded, y, test_size=0.2, random_state=42)

# ---- Architecture CNN 1D ----
model = models.Sequential([
    layers.Embedding(input_dim=5000, output_dim=64, input_length=30),
    layers.Conv1D(128, 5, activation='relu'),
    layers.MaxPooling1D(2),
    layers.Conv1D(64, 5, activation='relu'),
    layers.GlobalMaxPooling1D(),
    layers.Dense(64, activation='relu'),
    layers.Dense(5, activation='linear')
])
```

7. Moteur Géométrique : Blueprinting et YAML

Cette étape transforme les chiffres en géométrie.

- **Placement Spatial** : Le script divise une zone de 20m 15m.
- **Standardisation** : Les pièces sont alignées selon une logique de zones (Zone Jour / Zone Nuit).
- **Fichier YAML** : Sert de "mémoire morte" au robot, contenant chaque coordonnée (x, y) et les dimensions (w, h).

```
doors:
- orientation: horizontal
  x: 5.55
  y: 7.2
- orientation: horizontal
  x: 14.0
  y: 1.5
rooms:
- height: 7.5
  surface: 19.76626205444336
  type: salon
  width: 12.0
  x: 0
  y: 7.5
- height: 7.5
  surface: 19.060977935791016
  type: cuisine
  width: 8.0
  x: 12.0
  y: 7.5
- height: 6.0
  name: Chambre 1
  surface: 60.0
  type: chambre
  width: 10.0
```

8. Algorithme de Pilotage Initial : Go-to-Goal

Le premier contrôleur utilise la trigonométrie :

- $\text{Distance} = \sqrt{(x_{\text{goal}} - x)^2 + (y_{\text{goal}} - y)^2}$
- **Limites** : Ce modèle ne prend pas en compte les obstacles et souffre de dérives odométriques importantes.

9. Module IA 2 : Apprentissage par Renforcement (RL)

Dans le projet **AutoArchitect+**, le module d'apprentissage par renforcement est utilisé pour permettre au robot de tracer des lignes droites lors de la construction des murs, même lorsque les segments ne sont pas parfaitement alignés ou lorsqu'il y a de petites variations dans l'environnement. Pour cela, nous avons choisi **PPO (Proximal Policy Optimization)**, un algorithme réputé pour sa stabilité dans le contrôle continu des robots mobiles.

a. Choix de PPO

- **Stabilité** : PPO gère efficacement les mises à jour de la politique tout en évitant les comportements instables ou extrêmes.
- **Contrôle continu** : Il est particulièrement adapté pour des actions continues comme la vitesse linéaire et angulaire d'un robot mobile.
- **Simplicité d'intégration** : Compatible avec **Stable-Baselines3** et ROS, ce qui facilite l'entraînement et l'inférence dans notre environnement simulé.

b. Définition de l'environnement WallFollowEnv

L'environnement WallFollowEnv fait le lien entre **ROS** et **Stable-Baselines3**, simulant les interactions du robot avec les murs à construire :

- **Observation Space** : Un vecteur de 4 valeurs flottantes normalisées entre -1 et 1, représentant :
 - La distance moyenne au mur côté droit (distance_wall)
 - L'erreur d'angle (angle_error)
 - La vitesse linéaire (linear_vel)
 - La vitesse angulaire (angular_vel)
- **Action Space** : Un vecteur [v, w] correspondant à :
 - v : vitesse linéaire du robot
 - w : vitesse angulaire du robot
- **Callback ROS** : Les capteurs embarqués (LIDAR et odométrie) mettent à jour les valeurs de l'environnement en temps réel.

c. Fonction de récompense

La performance du robot est guidée par une **fonction de récompense** définie dans WallFollowEnv :

1. **Suivi du mur** : récompense maximale lorsque la distance au mur cible est respectée.
2. **Avancer droit** : bonus pour une vitesse linéaire positive.
3. **Trajectoire lisse** : pénalité si l'accélération angulaire varie trop, évitant les zigzags.

4. **Sécurité** : pénalité si le robot s'éloigne trop du mur ou risque une collision.

Cette ingénierie de récompense permet d'orienter l'apprentissage du robot vers des trajectoires droites et fluides.

d. Entraînement du modèle (train_ppo.py)

Le script train_ppo.py automatise l'entraînement du robot :

- Initialisation d'un **nœud ROS** pour synchroniser l'environnement et le robot simulé.
- Création d'un **environnement vectorisé** avec DummyVecEnv.
- Paramétrage du modèle PPO :
 - learning_rate = 1e-3
 - n_steps = 512
 - batch_size = 32
 - gamma = 0.95
 - total_timesteps = 120_000
- Sauvegarde du modèle entraîné sous wall_tracer_ppo.

L'entraînement permet au robot de corriger automatiquement ses trajectoires et de suivre un plan de construction avec précision.

a. Inférence et intégration (inference.py)

Pour utiliser le modèle entraîné dans la simulation, le script inference.py :

- Charge le modèle PPO pré-entraîné (wall_tracer_ppo).
- Récupère en temps réel les données des capteurs (/scan et /odom).
- Génère les commandes cmd_vel que le robot exécute pour suivre les murs.
- Publie un **chemin tracé** via le topic /trace_path pour visualiser la trajectoire.
- Arrête le robot en cas de collision détectée.

Cette orchestration permet au nœud house_tracer_node1.py d'utiliser le modèle RL pour tracer automatiquement des murs droits à partir d'un plan YAML

.

ANNEXE :

